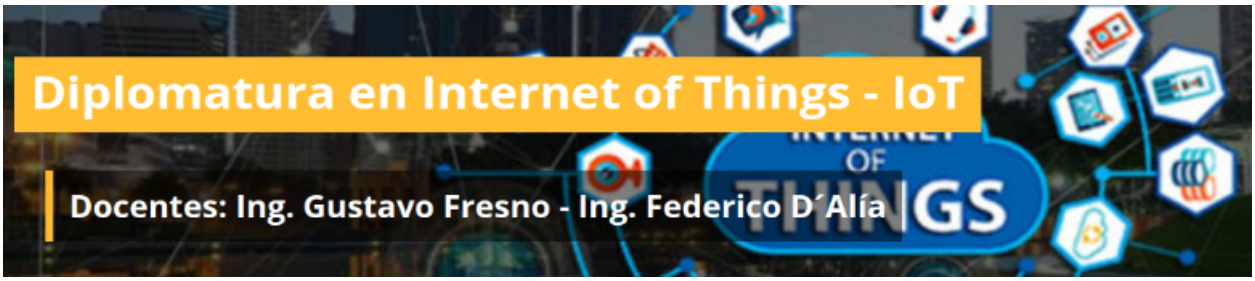
Diplomatura en Internet Of Things

ENTREGA TRABAJO PARCIAL INTEGRADOR - 2023

Esteban Andrés Fantini



Descripción del problemas/oportunidad de mejora.

La propuesta de mejora para este Trabajo Final de Proyecto (TPF) se centra en optimizar la gestión y producción de brotes de semillas en el vivero, que se encuentra en el patio trasero de la casa de campo del cliente. Mediante esta mejora, el cliente podrá realizar un monitoreo y control más eficiente de las condiciones ambientales y del suelo, mejorando así el crecimiento de los brotes de girasol.

Solución propuesta.

Para llevar a cabo esta optimización, escalaremos la solución anterior mediante la implementación del protocolo de comunicación MQTT, utilizando el servicio Mosquitto. Además, incorporaremos una placa Raspberry Pi en la configuración. En esta Raspberry Pi, instalaremos el servicio Mosquitto y Node-RED para facilitar la gestión de datos y su posterior análisis. La plataforma IOT seleccionada para la gestión del Internet de las cosas (IoT) será Blynk, la cual ofrece al cliente una interfaz intuitiva y accesible para el monitoreo remoto y control del vivero.

Arquitectura de la solución

Los sensores ubicados en el vivero (DHT22, TSL2561 y Capacitive Soil Moisture Sensor V1.2) capturan datos sobre las condiciones ambientales y del suelo. El ESP32, actuando como cliente MQTT, centraliza la adquisición de datos y los publica en tópicos específicos. La Raspberry Pi, configurada como broker MQTT mediante el servicio Mosquitto, recibe y canaliza estos datos hacia Node-RED, donde se lleva a cabo el procesamiento y análisis. Luego integraremos a la solucion la plataforma IOT Blynk, la cual nos permite la visualización remota y el control de los datos desde dispositivos móviles del cliente, proporcionando así una interfaz accesible.

Detalle del Hardware específico de la solución

- ***ESP32 Devkit V1****: Ubicado en el vivero, en modo STATION*
- ***Placa Raspberry Pi 4****: Ubicado en el vivero. Actuará como un broker MQTT utilizando Mosquitto y aloja Node-RED para el procesamiento y análisis de datos.*
- ***Extensor de Rango WiFi****: Ubicado en el vivero. Ampliará la cobertura del WiFi para garantizar una conexión estable entre los dispositivos del vivero (ESP32, Raspberry Pi) y el router de la casa.*
- ***Cable Ethernet****: El extensor de rango se conecta mediante un cable Ethernet al router de la casa y del mismo modo se conecta el router al modem.*
- ***Router de la casa****: Establece la conexión con el extensor de rango via cable ethernet.*
- ***Modem****: de la conexión a internet*
- ***Cable modem o ADSL****: conectado al router, de conexión a internet.*

Conectividad del Hardware

- ***WiFi****: Se utiliza para la comunicación inalámbrica entre todos los dispositivos en el vivero, incluyendo ESP32, Raspberry Pi y el extensor de rango.*
- ***Ethernet****: El extensor de rango se conecta mediante un cable Ethernet al router, estableciendo así la conexión entre los dispositivos del vivero y la red local.*

Detalle de los sensores específicos de la solución.

- *Para controlar los parámetros Temperatura y Humedad:* ***Sensor DHT22***
- *Para controlar el parámetro Luminosidad:* ***Sensor TSL2561***
- *Para controlar el parámetro Humedad del suelo:* ***Capacitive Soil Moisture Sensor V1.2***

Detalle de los actuadores específicos de la solución

- *Para el Sistema de riego:* ***Relé 5V 10A 220V*** *que controla la válvula solenoide.*
- *Para el Inyector de aire:* ***Relé 5V 10A 220V*** *que controla el motor del inyector.*
- *Para el Extractor de aire:* ***Relé 5V 10A 220V*** *que controla el motor del extractor.*

Detalle de la plataforma a utilizar.

- *Se utilizará la plataforma IOT "****BLINK****". Se debe crear un proyecto en la aplicación Blynk y configurar widgets para visualizar datos y controlar actuadores.*

Diagrama funcional por bloques

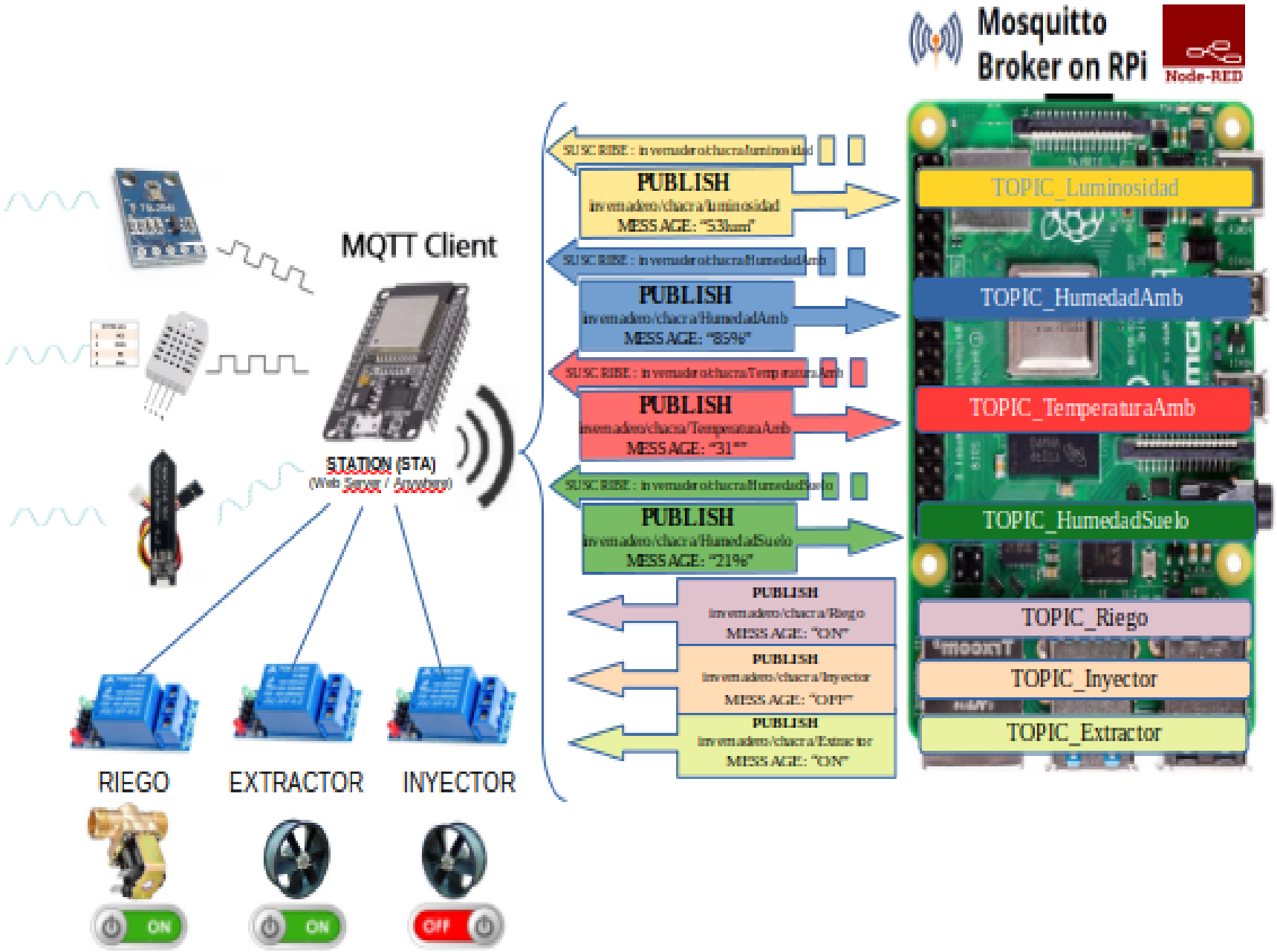
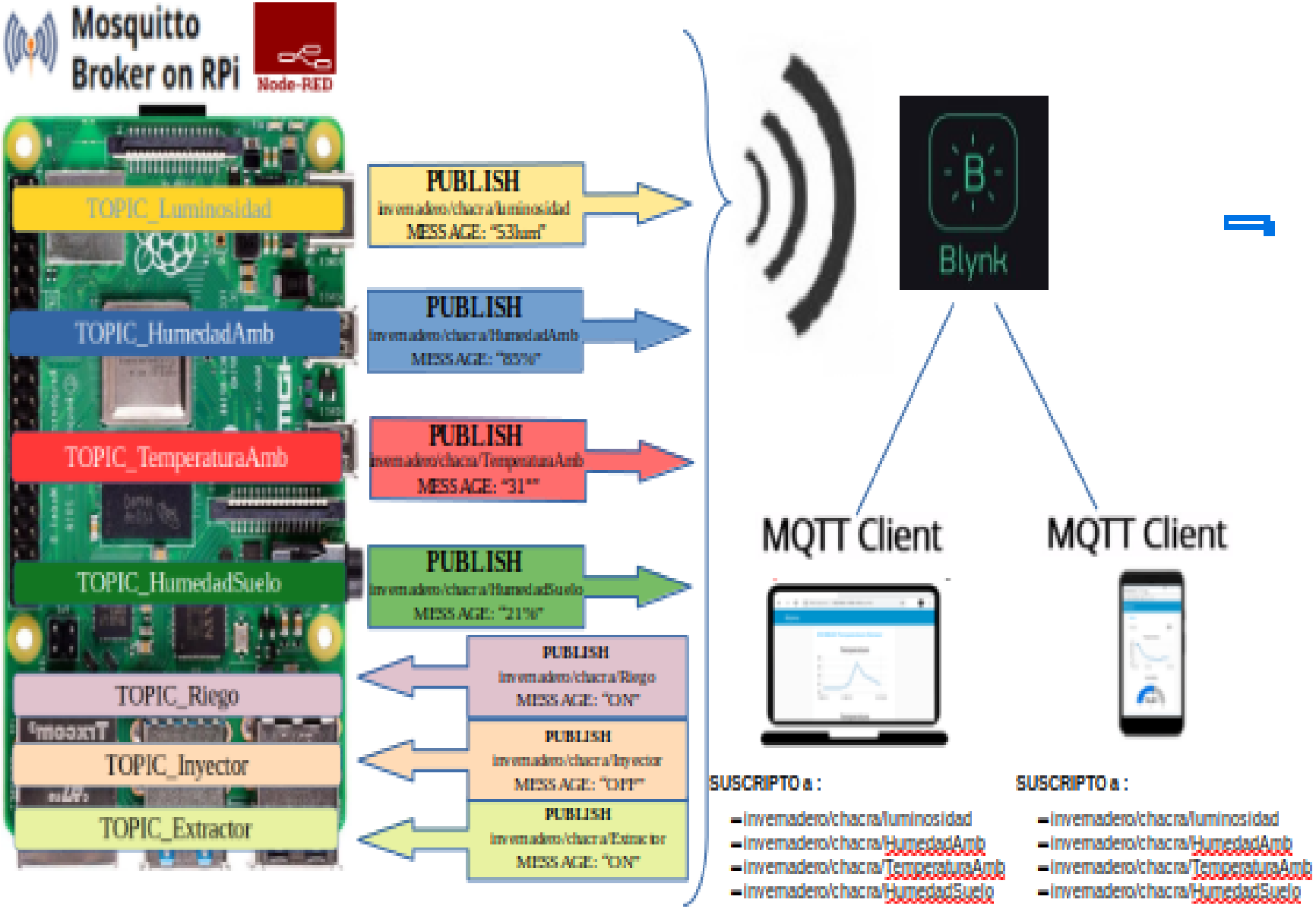
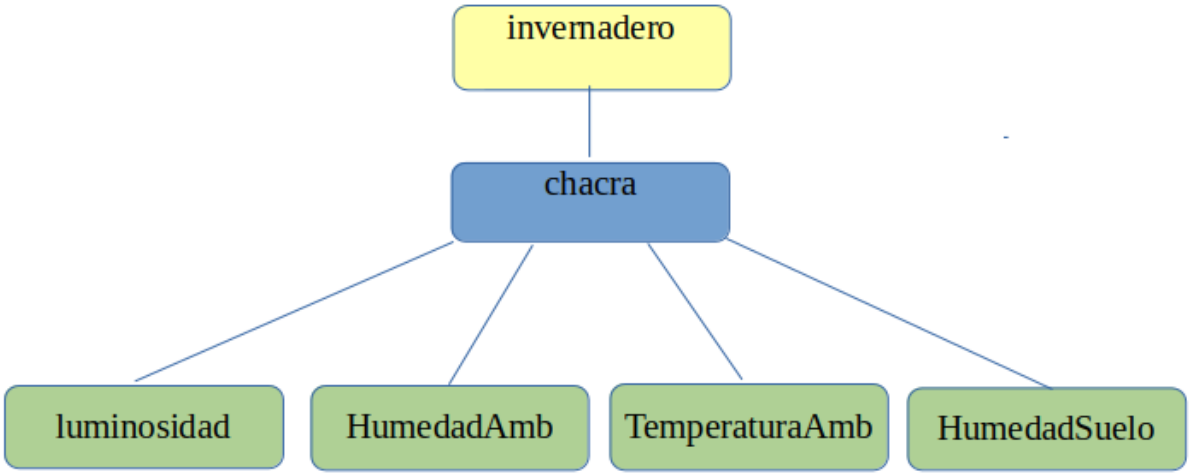

Mosquitto
Broker on RPi
Node-RED
MQTT Client
STATION (STA)
(Web Server / Anywhere)
SUSCRIBE : invernadero/chacra/luminosidad
PUBLISH
invernadero/chacra/luminosidad
MESSAGE: "53lum"
SUSCRIBE : invernadero/chacra/HumedadAmb
PUBLISH
invernadero/chacra/HumedadAmb
MESSAGE: "85%"
SUSCRIBE : invernadero/chacra/TemperaturaAmb
PUBLISH
invernadero/chacra/TemperaturaAmb
MESSAGE: "31°"
SUSCRIBE : invernadero/chacra/HumedadSuelo
PUBLISH
invernadero/chacra/HumedadSuelo
MESSAGE: "21%"
PUBLISH
invernadero/chacra/Riego
MESSAGE: "ON"
PUBLISH
invernadero/chacra/Inyector
MESSAGE: "OFF"
PUBLISH
invernadero/chacra/Extractor
MESSAGE: "ON"
TOPIC_Luminosidad
TOPIC_HumedadAmb
TOPIC_TemperaturaAmb
TOPIC_HumedadSuelo
TOPIC_Riego
TOPIC_Inyector
TOPIC_Extractor
RIEGO
EXTRACTOR
INYECTOR
ON
ON
OFF

DIAGRAMA Continuación...



Jerarquía de Niveles de TÓPICOS MQTT



Ruta del flujo de datos desde el vivero el teléfono móvil.

El flujo de datos recolectados por los sensores sigue una ruta específica desde el vivero hasta la visualización en tiempo real y el control remoto en la aplicación Blynk del teléfono móvil. Tomando como ejemplo el sensor de temperatura, DHT22, que mide la temperatura en el vivero, el flujo del dato inicia con la generación de un valor por parte del sensor en cuestion. Este dato es adquirido por el ESP32 configurado como cliente MQTT, que lo publica en un tópico MQTT designado para la temperatura. La Raspberry Pi, actuando como broker MQTT, recibe estos datos al estar suscrita al tópico correspondiente. Node-RED, presente en la Raspberry Pi, procesa y analiza la información según la lógica programada. A continuación, los datos pueden ser publicados en un tópico específico para Blynk desde Node-RED, o Blynk puede suscribirse directamente al tópico de temperatura MQTT. La aplicación Blynk en el teléfono móvil se conecta a la plataforma Blynk, recibiendo datos del tópico específico y actualizando la interfaz de usuario en tiempo real con la nueva temperatura. Así, la visualización en el teléfono móvil proporciona al usuario una representación actualizada y accesible de las condiciones de temperatura en el vivero, ofreciendo la posibilidad de configurar widgets adicionales para alertas o acciones específicas (control de los actuadores)

según los valores medidos. Este proceso integral garantiza una monitorización efectiva y un control remoto conveniente del entorno del vivero.

Activación de actuadores desde el móvil

El usuario visualiza la temperatura actualizada en la aplicación Blynk del teléfono móvil y según los valores de temperatura puede tomar la decisión de activar o no los actuadores..

Cuando el usuario decide activar la válvula solenoide que controla el sistema de riego, la aplicación Blynk envía la instrucción correspondiente a la plataforma Blynk. Desde allí, se puede utilizar un nodo en Node-RED para publicar los datos procesados en un tópico específico de MQTT, o Blynk puede estar directamente suscrito al tópico de control de MQTT. La Raspberry Pi, actuando como broker MQTT con el servicio Mosquitto, recibe la instrucción de la aplicación Blynk al estar suscrita al tópico MQTT correspondiente a la activación de la válvula solenoide. Posteriormente, el ESP32, configurado como cliente MQTT, también está suscrito a este tópico y recibe la instrucción de activar la válvula solenoide. Finalmente, el ESP32 que controla la válvula solenoide en el vivero envía tensión al relé que la abre permitiendo el paso del agua.

Códigos

1) Software cargado en la placa ESP32 devkit-v1

```
// Código para monitorear la temperatura y la humedad usando el tablero node rede
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

// Configuración de la red WiFi
const char* ssid = "Mi_WiFi";  // "Wokwi-GUEST"
const char* password = "secreta";
// Dirección del servidor MQTT (en este caso, se utiliza un broker público para pruebas)
const char* mqtt_server = "test.mosquitto.org"; // MQTT broker

// Tópicos MQTT para temperatura y humedad
char* my_topic2 = "/invernadero/chacra/TemperaturaAmb"; // Tópico 1: temperatura del sensor
char* my_topic3 = "/invernadero/chacra/HumedadAmb"; // Tópico 2: humedad Registrada

// Cliente para la conexión WiFi y MQTT
WiFiClient espClient;
PubSubClient client(espClient);

//// Configuración del sensor de temperatura y humedad (DHT22)
DHT dht(4, DHT22);
```

```
void setup() {
  Serial.begin(115200); // Inicializamos la comunicación serial

// Inicialización de la conexión WiFi
  setup_wifi();

// Configuración del servidor MQTT, del puerto del broker mqtt y del sensor DHT22
  client.setServer(mqtt_server, 1883);
   dht.begin();
   delay(2000);
}
// Conectándome a la red wi-fi usando las credenciales proporcionadas
void setup_wifi() {
 delay(10);
 Serial.println();
 Serial.print("Connecting to ");
 Serial.println(ssid);

 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
 }
 Serial.println("");
 Serial.println("WiFi connected");
 Serial.println("IP address: ");
```

```
  Serial.println(WiFi.localIP()); // Imprime la dirección y p local del esp32 una vez conectado
}

// Función para reconectar al servidor MQTT en caso de pérdida de conexión
void reconnect() {
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    if (client.connect("ESPClient")) {
      Serial.println("connected");
      client.subscribe(my_topic2);
      client.subscribe(my_topic3);
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
      }
   }
}
void loop() {
  // Verificación y procesamiento de mensajes MQTT
  if (!client.connected()) {
    reconnect();
  }
  client.loop();
  delay(100); // Agrega un retraso para evitar inundar el broker mqtt con mensajes
  float temp = dht.readTemperature();
```

```
  float humidity = dht.readHumidity();
  Serial.print("Temp : ");
  Serial.print(temp);
  Serial.println(" C ");
  Serial.print("Humidity : ");
  Serial.print(humidity);
  Serial.println(" % ");
  String message_temp = String(temp);
  String message_hum = String(humidity);

// Publicación de datos en los tópicos MQTT
  client.publish("/invernadero/chacra/TemperaturaAmb", messaget.c_str());
  client.publish("/invernadero/chacra/HumedadAmb", messageh.c_str());
  delay(200);
}
```

2) Software para configurar la Raspberry Pi

(Raspberry Pi como un broker MQTT utilizando el servicio Mosquitto y suscribipcion a los tópicos mencionados en el script del ESP32)

- Instalamos Mosquitto instalado en tu Raspberry Pi.

```
sudo apt update
```

```
sudo apt install -y mosquitto mosquitto-clients
```

- Utilizaremos la biblioteca paho-mqtt para la comunicación MQTT.

```
pip install paho-mqtt
```

Script Python:

```
import paho.mqtt.client as mqtt


# Configuración del broker MQTT
mqtt_broker = "test.mosquitto.org"  # Cambia a tu dirección IP si es local
mqtt_port = 1883


# Tópicos a los que se suscribirá la Raspberry Pi
topic_temp = "/invernadero/chacra/TemperaturaAmb"
topic_hum = "/invernadero/chacra/HumedadAmb"


# Función que se ejecuta cuando se recibe un mensaje en algún tópico
```

```
def on_message(client, userdata, message):
    print(f"Mensaje recibido en el tópico {message.topic}: {message.payload}")

# Configuración del cliente MQTT
client = mqtt.Client()
client.connect(mqtt_broker, mqtt_port, 60)

# Asociación de la función on_message al evento de recibir mensaje
client.on_message = on_message

# Suscripción a los tópicos de temperatura y humedad
client.subscribe(topic_temp)
client.subscribe(topic_hum)

# Mantener la Raspberry Pi a la espera de mensajes
client.loop_forever()
```